

1. List

Browser History Manager (Doubly Linked List)

Objective

Implement a **custom browser history manager** using a **Doubly Linked List**.

Your implementation should allow users to visit new pages and navigate **backward and forward** within their browsing history.

Requirements

- visitPage(url: string): void**
 - Adds a new page to the history.
 - If a new page is visited after going back, remove all forward history.
- goBack(): string | null**
 - Moves back to the previous page in history.
 - Returns **null** if there is no previous page.
- goForward(): string | null**
 - Moves forward to the next page in history.
 - Returns **null** if there is no next page.
- getCurrentPage(): string | null**
 - Returns the URL of the current page.

Constraints

- Must use a **Doubly Linked List** for history storage.
- No built-in arrays or lists allowed for storing history directly.

Implementation (React + TypeScript)

```
import { useState } from "react";

/**
 * Represents a node in the doubly linked list.
 */
class ListNode {
  url: string;
  prev: ListNode | null = null;
  next: ListNode | null = null;

  constructor(url: string) {
    this.url = url;
  }
}

/**
 * Manages browser history using a doubly linked list.
 */
class BrowserHistory {
  private current: ListNode | null = null;

  /**
   * Visits a new page and clears forward history.
   */
  visitPage(url: string) {
    const newNode = new ListNode(url);

    if (this.current) {
      this.current.next = null; // Clear forward history
      newNode.prev = this.current;
      this.current.next = newNode;
    }

    this.current = newNode;
  }

  /**
   * Moves back in history.
   */
  goBack(): string | null {
    if (this.current?.prev) {
      this.current = this.current.prev;
      return this.current.url;
    }

    return null;
  }

  /**
   * Moves forward in history.
   */
  goForward(): string | null {
    if (this.current?.next) {
      this.current = this.current.next;
      return this.current.url;
    }

    return null;
  }

  /**
   * Returns the current page URL.
   */
  getCurrentPage(): string | null {
    return this.current ? this.current.url : null;
  }
}

/**
 * React component for managing browser history.
 */
const BrowserHistoryComponent = () => {
  const [history] = useState(() => new BrowserHistory());
  const [currentPage, setCurrentPage] = useState<string | null>(null);

  const visitPage = () => {
    const newPage = `Page ${Math.floor(Math.random() * 100)}`;
    history.visitPage(newPage);
    setCurrentPage(history.getCurrentPage());
  };

  const goBack = () => {
    setCurrentPage(history.goBack());
  };

  const goForward = () => {
    setCurrentPage(history.goForward());
  };

  return (
    <div className="p-4 border rounded shadow-lg w-96">
      <h2 className="text-xl font-bold"> Custom Browser History</h2>
      <div className="mt-2">
        <p className="text-sm text-gray-500">Current Page:</p>
        <p className="text-lg font-semibold">{currentPage || "No Page Yet"}</p>
      </div>

      <div className="mt-4 flex gap-2">
        <button
          onClick={visitPage}
          className="px-3 py-1 bg-blue-500 text-white rounded"
        >
          Visit New Page
        </button>
        <button
          onClick={goBack}
          className="px-3 py-1 bg-gray-400 text-white rounded"
        >
          Back
        </button>
        <button
          onClick={goForward}
          className="px-3 py-1 bg-gray-400 text-white rounded"
        >
          Forward
        </button>
      </div>
    </div>
  );
};

export default BrowserHistoryComponent;
```

Infinite Image Carousel (Circular Linked List)

Objective

Build an **infinite scrolling image carousel** using a **Circular Linked List**.

Users should be able to navigate between images seamlessly, where the last image loops back to the first one.

Requirements

- addImage(url: string): void**
 - Adds a new image to the carousel.
- nextImage(): string**
 - Moves to the next image in the sequence.
 - Loops back to the first image if at the last image.
- prevImage(): string**
 - Moves to the previous image in the sequence.
 - Loops back to the last image if at the first image.
- getCurrentImage(): string | null**
 - Returns the current image URL.

Constraints

- Must use a **Circular Linked List** to store images.
- No built-in arrays or lists allowed for storing images directly.

LRU Cache for API Responses (LRU Cache with Doubly Linked List + HashMap)

Objective

Implement a **Least Recently Used (LRU) Cache** to store API responses.

The cache should remove the **least recently accessed data** when full.

Requirements

- put(key: string, value: string): void**
 - Stores a key-value pair in the cache.
 - If the cache is full, removes the **least recently used item** before adding the new item.
- get(key: string): string | null**
 - Retrieves the value associated with a given key.
 - If the key is not found, return **null**.
 - Marks the accessed key as **most recently used**.
- size(): number**
 - Returns the current size of the cache.

Constraints

- Must use a **Doubly Linked List** and **HashMap** for efficient cache management.
- No built-in Map, Set, or caching libraries allowed.

Social Media Activity Feed (Doubly Linked List)

Objective

Design a **social media activity feed system** using a **Doubly Linked List**.

The system should maintain a list of recent activities and allow users to navigate through them.

Requirements

- addActivity(activity: string): void**
 - Adds a new activity to the feed (most recent first).
- deleteActivity(index: number): void**
 - Removes an activity at the specified index.
- showActivities(): string[]**
 - Returns all activities in order (most recent first).

Constraints

- Must use a **Doubly Linked List** to store activities.
- No built-in arrays or lists allowed for storing activities directly.

